

Corrupted by Thanos's Attack

MIRAGE 2.0 — Forensics — Analysis Status Report (Unresolved)

CATEGORY

Forensics

FILES

thanos_firmware_dump.bin (two variants)

DEPLOYMENT

Offline analysis — no execution required

VARIANTS

Corrupted_by_Thanos_Attack.zip,
thanos_keks.zip

STATUS

UNRESOLVED — flag not recovered

NOTE: This document is an honest account of the investigation, not a solved writeup. Despite two independent, deep analysis passes, the final flag for either variant of this challenge was **not** recovered. It's written up step by step anyway so the organizer can see exactly where the trail runs out, compare it against their own reference solution, and either patch the challenge or point out what was missed.

Step 1 — First Look

Both zips ship one file, `thanos_firmware_dump.bin`, described as a damaged firmware image. `file` reports it as raw data with no recognized container format, so the first move is a straight hex dump rather than trusting any tool to auto-identify it:

```
$ xxd thanos_firmware_dump.bin | head
00000000: 5448 4e58 0300 0000 ea08 0000 5448 414e  THNX.....THAN
00000010: 4f53 2d4c 4142 2d37 4633 4100 0400 0000  OS-LAB-7F3A.....
```

A 4-byte ASCII magic (THNX) at the very start of a “damaged” file is a strong signal that the damage is content-level, not structural — the container format itself is intact and worth reverse engineering properly before assuming anything is actually corrupted.

Step 2 — Cracking the Container Format

Working outward from the magic bytes, and cross-checking small integer fields against the actual file size (a size field reading exactly 2282 against a 2282-byte file is not a coincidence), the header structure falls out:

```
offset 0x00  magic      'THNX'              (4 bytes)
offset 0x04  version    u32 LE
offset 0x08  total_size u32 LE  (== file size, in the clean variant)
offset 0x0c  label      16 bytes, NUL-padded ASCII
offset 0x1c  num_sections u32 LE
offset 0x20  section table: N x 20-byte entries:
                name[8], offset(u32), size(u32), crc32(u32)
```

Each section entry ends in what looks like a checksum; recomputing `zlib.crc32()` over the bytes each entry points at and comparing against the stored value is a cheap, mechanical way to find out which sections are

trustworthy — and confirms the format guess, since every computed value matches immediately.

Step 3 — The “Clean” Variant

Parsing `Corrupted_by_Thanos_Attack.zip`'s dump with this format gives four sections — `BOOT`, `LOGS`, `CONFIG`, `RECOVERY` — **all four CRC32-valid**. Reading `CONFIG`'s text turns up the line `ARCHIVE=RECOVERY`, pointing directly at the `RECOVERY` section (202 bytes) as the thing to actually decode. Measuring its byte distribution shows Shannon entropy around 6.9 out of a possible 8 — statistically close to random, with no plaintext visible under `strings`.

From there, the natural next question is “encoded how?”, and each candidate answer was tried and ruled out in turn:

- `binwalk` against the whole image and the isolated blob — zero embedded-file signatures detected.
- Raw-deflate / bz2 / lzma decompression attempted at every possible starting byte offset, in case the compressed stream doesn't begin at byte 0.
- Single-byte XOR / ADD / SUB brute force, scoring each candidate by what fraction of the result is printable ASCII.
- 50+ multi-byte XOR and RC4 key candidates built from everything else visible in the file: the container's label, the “`RECOVERY_SALT`” string seen in the other variant, section names, and MD5/SHA1/SHA256 digests of all of the above, tried forwards and reversed.
- Classical repeating-key-XOR cryptanalysis (Kasiski examination / index-of-coincidence) across key lengths 1–60, in case the key isn't derived from anything visible in the file at all.
- Bit-level transforms — nibble swap, bit reversal, atbash, high-bit stripping — in case the “encryption” is simpler than a keyed cipher.
- xorshift32-style PRNG keystreams seeded from header fields (version, size, the section CRCs), in case the key is generated rather than stored.
- Two-time-pad checks against the high-entropy tail bytes found in the `BOOT/LOGS/CONFIG` sections — each of those has a readable ASCII header followed by a long high-entropy tail, confirmed genuinely random rather than a repeated fill byte, raising the question of whether it was XORed with the same keystream as `RECOVERY`. No shared key recovered from the overlap either.
- Inter-section gap bytes — confirmed to be pure alignment padding once examined directly, nothing hidden there.

None of these produced anything resembling a flag. At this point the “clean” variant's trail runs out.

Step 4 — The “Trap” Variant

`thanos_keks.zip`'s dump starts identically, but parsing it with the same fixed offsets breaks: the header is deliberately mangled, with a spliced-in 20-byte string (`SNAP-RECOVERY-19`) sitting exactly where `num_sections` and the start of the section table should be. Rather than trusting the header at all, locating section names directly via `strings -t x` and re-deriving a 20-byte-stride table by hand recovers **nine** section entries — and, unexpectedly, **all nine are CRC32-valid**. That's an important finding on its own: CRC validity, which worked as a trust signal on the clean variant, does not distinguish real data from decoys in this one.

Four of the nine sections (`BOOTLOG`, `TRACE`, `CONFIG`, `CACHE`) contain readable log/config text, each immediately followed by **two** plaintext strings shaped exactly like flags and then a run of single-byte padding.

That's eight flag-shaped strings in total, all sitting inside CRC-valid sections — and all eight are the same length and character set, with nothing (hash, checksum, position) marking any one of them as more genuine than the others. Combined with the README's own warning (“do not trust a flag merely because strings finds it”), the working assumption became that all eight are bait, not that one of them is secretly real.

Step 5 — Chasing the Remaining Lead

The other five sections have short, meaningless four-character names (X7Q2, N4KP, Z9LM, R2VX, H8DS) and high-entropy contents — the strongest remaining candidate for where a real hidden payload might live, on the theory that meaningless names are what a genuinely hidden section would have, versus the deliberately readable names used for the bait sections. These were tried individually, concatenated in several different orders, and cross-keyed against the clean variant's RECOVERY blob — all unsuccessful. One structural oddity was noted but never turned into a working decode: Z9LM is exactly 32 bytes (the length of a SHA-256 digest) versus the other four sections' 36 bytes, and is the only one of the five with no 0xa5 filler gap immediately after it — worth investigating further if picked back up.

Where the Trail Runs Out

Given the volume of techniques ruled out across two independent passes on both variants, this likely needs one of: (a) a hint or technique that wasn't considered, (b) a fix to the packaging so the intended decode path actually works, or (c) a stronger nudge in the README toward the intended tool or technique. Recommend testing this pair against your own reference solve script before the event goes live.